PESUPool: An Intelligent Database-Driven Carpooling Platform

DBMS Experiential Learning – Level 3 Capstone Project Report

PES University, Bengaluru | Dept of Computer Science & Engineering | Instructor: Dr. Nagasundari S

Team Member	SRN	Email	Project Contribution Scope
Team Member 1	**PES1UG24AM121**	pes1ug24am121@pesu.pes.edu	Relational Schema Design, 3NF Normalization, DDL/DML, ACID Stored Procedures (sp_book_ride), 5 Database Triggers, Indexing
Team Member 2	**PES1UG24AM078**	pes1ug24am078@pesu.pes.edu	Next.js 14 Web Interface, REST Route Handlers, MongoDB Telemetry Stream, Aggregation Pipeline, Analytics, Documentation

1. Problem Statement & Motivation

Urban transit across Bengaluru is characterized by chronic vehicular congestion, peak-hour bottlenecks, long commute durations, high fuel expenses, and significant carbon emissions. For students and faculty members of PES University, traversing between the Ring Road (RR) Campus, Electronic City (EC) Campus, and major public transport nodes (Banashankari Metro, Silk Board Junction, and Koramangala) represents a daily logistical challenge.

Commercial ride-hailing services (Uber/Ola) impose aggressive surge pricing during college rush hours and lack campus-level trust verification. Simultaneously, hundreds of private vehicles commute to PES University campuses every day with multiple vacant seats. PESUPool provides a centralized, database-driven carpooling platform tailored specifically to the PES University community, uniting relational ACID guarantees with high-throughput NoSQL telemetry.

2. Project Objectives & Scope

- **ACID Seat Reservation:** Implement row-level locking (SELECT ... FOR UPDATE) inside stored procedures to guarantee that concurrent seat reservation attempts never exceed vehicle capacity.
- **Campus Identity Verification:** Enforce institutional email verification (@pes.edu / @pesu.pes.edu) and valid SRN/PRN strings.
- **Multi-Paradigmatic Persistence:** Relational storage for financial ledgers and reservations (MySQL 8.4); document store for high-frequency GPS breadcrumbs (MongoDB).
- **Cost Splitting & Carbon Offset:** Intelligently calculate per-seat travel expenses based on route distance and fuel classification, tracking cumulative carbon emission reductions (~0.171 kg CO2 avoided per passenger-km).
- **Deep DBMS Analytics:** Provide analytical reports executing multi-table joins, GROUP BY, HAVING, and subqueries.

3. System Architecture & Database Paradigm Justification

Relational SQL (MySQL 8.4): Core business entities require strict ACID compliance, referential integrity, and row-level concurrency protection. Tables for users, vehicles, routes, rides, bookings, and payments require exact foreign key enforcement (ON DELETE CASCADE) to prevent data anomalies.
Document NoSQL (MongoDB): Moving carpool vehicles generate frequent GPS breadcrumbs, speed metrics, and in-transit messaging. Writing this high-velocity time-series stream into relational tables would cause excessive lock contention. MongoDB handles schema-free telemetry appends efficiently and powers a MongoDB Aggregation Pipeline ($group, $avg, $sort).

4. Entity-Relationship (ER) Modeling (10 Entities in 3NF)

The relational database schema is normalized up to **Third Normal Form (3NF)** and contains 10 base tables (exceeding the course requirement of 5):
1. **USERS:** Core entity storing validated SRN/PRN, @pes.edu email, role, campus, department, phone, and emergency contact.
2. **DRIVERS:** 1:1 user specialization tracking license numbers, expiry dates, verified status, and lifetime ratings.
3. **PASSENGERS:** 1:1 user specialization tracking default pickup points, lifetime rides, and prepaid PESU Wallet balances.
4. **VEHICLES:** 1:N with drivers, storing vehicle make, model, registration plate, fuel type, and seating capacity.
5. **ROUTES:** University transit corridors with origin, destination, intermediate pickup points JSON, distance in km, and base fare.
6. **RIDES:** Scheduled carpool trips linked to driver, vehicle, and route, tracking date, departure time, and available seats.
7. **BOOKINGS:** Confirmed seat reservations with boarding checkpoints and cryptographic QR verification tokens.
8. **PAYMENTS:** 1:1 with bookings, tracking payment method (PESU_WALLET, UPI, CASH), amount, and transaction references.
9. **RATINGS:** Two-way rating matrix capturing star scores, punctuality, and driving safety feedback.
10. **CANCELLATIONS:** Audit ledger tracking cancellations, penalty fees, and automated passenger wallet refunds.

5. Advanced Database Concepts Demonstrated (6 Features)

- **ACID Transactions (sp_book_ride):** Uses START TRANSACTION, SELECT ... FOR UPDATE, and ROLLBACK handlers to eliminate double-booking.
- **Database Views:** vw_available_rides, vw_driver_performance, vw_route_metrics, and vw_user_carbon_savings.
- **Database Triggers:** 5 active triggers enforcing self-booking prevention, overbooking capacity checks, seat decrements, cancellation refunds, and dynamic rating recalculations.
- **Stored Procedures:** Modular encapsulated routines: sp_book_ride, sp_cancel_booking, and sp_generate_campus_report.
- **Composite B-Tree Indexing:** Indexes on (travel_date, status, departure_time) and (origin_campus, destination_campus).
- **NoSQL Aggregation Pipeline:** Live MongoDB aggregation calculating segment speed averages and traffic delays.

6. Key Analytical Queries & Demonstration Reports

1. Department Mobility & Financial Savings: Multi-table join across users, passengers, bookings, rides, and routes grouped by department. Computes actual expenses vs commercial cabs (■22/km benchmark) and CO2 avoided (0.171 kg/km).
Results: ECE saved ■1,080.40 (10.8 kg CO2); AIML saved ■898.00 (8.4 kg CO2); Mechanical saved ■652.20 (6.9 kg CO2).
2. Route Occupancy & Revenue: Multi-table join with HAVING COUNT(b.booking_id) > 0 evaluating route loads.
3. Peak Commute Departure Hours: SQL date/time extraction HOUR(r.departure_time) showing 08:00-09:00 AM as peak volume window.
4. Driver Safety Leaderboard: Multi-join filtering drivers with HAVING overall_avg_rating >= 4.50.
5. Nested Subquery: Filters drivers whose total earnings exceed the campus average earnings using nested aggregate subqueries.

7. Testing Results & Integrity Verification

Test ID	Test Case Scenario	Expected System Behavior	Observed Status
TC-01	Atomic Seat Reservation via sp_book_ride	Deduct wallet, lock row, create pass, decrement seats	**PASS**
TC-02	Driver Self-Booking Prevention Trigger	Abort insertion: Driver cannot book own ride	**PASS**
TC-03	Overbooking Prevention Trigger	Abort transaction: Insufficient seats remaining	**PASS**
TC-04	Cancellation & 90% Wallet Refund	Restore seats, deduct 10% fee, refund wallet	**PASS**
TC-05	NoSQL Real-Time Telemetry Stream	Append GPS coordinates to MongoDB without locks	**PASS**

8. Conclusion

PESUPool successfully applies core database concepts—including relational ER design, 3NF normalization, ACID transaction processing, database triggers, views, and NoSQL telemetry streams—to solve an acute daily commuter problem for PES University. The platform provides a scalable foundation for campus carpooling while demonstrating rigorous database engineering principles.